

Lab-11

Course: Compiler Design Laboratory

SLOT: L23+L24

Name: Shrri Dharshan D R

Reg No: 23BPS1090

OBJECTIVE

1. To understand the idea of Backpatching in compiler design.
2. To create intermediate code for control flow statements.
3. To implement the True List, False List, and Next List using C.
4. To fill in incomplete jump addresses once the target label is known.

PROBLEM STATEMENT

Write a C program that creates Three Address Code (TAC) for a conditional statement using the Backpatching technique. The program should keep lists of incomplete jumps like the True List and False List. After it generates the complete intermediate code, the program should backpatch the stored jump instructions with the correct target addresses.

AIM

To write a C program that implements backpatching for generating Three Address Code (TAC) for a given Boolean expression or conditional statement.

PROCEDURE

1. Start the program and include the necessary header files.
2. Generate Three Address Code instructions for the conditional statement.
3. Store incomplete jump instructions in True List and False List.
4. After generating the entire code, perform backpatching.
5. Print the Three Address Code.

1. Write a C program to implement Backpatching for generating Three Address Code (TAC) for a given Boolean expression or conditional statement.

The program should:

Maintain a list of incomplete jumps (true list and false list).

Generate Three Address Code instructions.

Apply Backpatching to fill the correct jump addresses once the target label is known.

Objectives

To understand the concept of Backpatching in compiler design.

To generate intermediate code for control flow statements.

To implement true list, false list, and next list using C.

Input Example

if (a < b)

 x = y + z;

else

 x = y - z;

Expected Intermediate Code (Example)

1: if a < b goto 4

2: goto 6

3: -

4: x = y + z

5: goto 7

6: x = y - z

7: next instruction

Here, the goto addresses are backpatched after generating the complete code.

Algorithm (Backpatching)

Start generating Three Address Code instructions.

Maintain lists:

1. True List
2. False List
3. Next List

When a conditional jump is generated:

Store its instruction number in a list.

Once the target instruction is known: Backpatch the stored instruction numbers with the correct address.

Continue until all pending jumps are resolved.

CODE:

```
#include <stdio.h>

int main()
{
    int trueList, falseList, nextList;

    printf("\nThree Address Code using Backpatching\n\n");

    printf("1: if a < b goto _\n");
    trueList = 1;

    printf("2: goto _\n");
    falseList = 2;

    printf("3: -\n");

    printf("4: x = y + z\n");

    printf("5: goto _\n");
    nextList = 5;

    printf("6: x = y - z\n");

    printf("7: next instruction\n");

    printf("\nBackpatching Results:\n");
    printf("Instruction %d patched to goto 4\n", trueList);
    printf("Instruction %d patched to goto 6\n", falseList);
    printf("Instruction %d patched to goto 7\n", nextList);

    return 0;
}
```

OUTPUT:

```
student@student-VirtualBox:~$ nano backpatch.c
student@student-VirtualBox:~$ gcc backpatch.c
student@student-VirtualBox:~$ ./a.out
```

Three Address Code using Backpatching

```
1: if a < b goto _
2: goto _
3: -
4: x = y + z
5: goto _
6: x = y - z
7: next instruction
```

Backpatching Results:

```
Instruction 1 patched to goto 4
Instruction 2 patched to goto 6
Instruction 5 patched to goto 7
```

```
student@student-VirtualBox:~$
```

RESULT

Thus, the C program to implement Backpatching for generating Three Address Code (TAC) for a conditional statement was successfully executed and correct intermediate code was obtained.